CAMPUSCONNECT

Student and Campus Event Management System

Programming in Java — Project Report

Student	Varanasi Mourya
Registration Number	25BAI10386
Course	Programming in Java
Technology	Java 17, Maven, JUnit Jupiter
Project type	Console-based Java application
Submission	VITyarthi project report

Prepared for academic project evaluation

1. Introduction

CampusConnect is a console-based Java application that organizes student registration, campus event creation, seat booking and cancellation, booking lookup, and summary reporting. It demonstrates object-oriented programming, modular package organization, input validation, automated testing, and Maven-based build management. It is a classroom-scale project, not a production university information system.

2. Problem Statement

Student records and event participation are often maintained in separate spreadsheets or manual lists. These methods can cause duplicate records, inconsistent seat counts, difficult cancellation tracking, and slow reporting. CampusConnect brings these activities into a structured application workflow.

3. Objectives

• Register students and assign unique identifiers.

• Create campus events with date, venue, and capacity.

• Book and cancel event seats while maintaining availability.

• View bookings and generate student/event reports.

• Apply modular Java design, validation, testing, and Maven packaging.

• Support local persistence and verify it across application restarts.

4. Functional Requirements

ID	Requirement
FR-01	Register a student and generate an ID.
FR-02	List student records.
FR-03	Create events with date, venue, and capacity.
FR-04	List events and available seats.
FR-05	Book a seat for a registered student.
FR-06	Cancel a booking.
FR-07	View booking records.
FR-08	Generate student and event reports.
FR-09	Validate input and display useful errors.
FR-10	Save and reload records across restarts.

5. Non-Functional Requirements

• Usability: clear menu options and prompts.

• Maintainability: separate UI, services, and model responsibilities.

• Reliability: keep booking state and seat counts consistent.

• Portability: Java 17+ and Maven.

• Performance: responsive for a classroom-sized dataset.

• Error handling: report invalid input and storage failures.

6. System Architecture

The console collects input and displays results. Services implement business operations. Model classes represent student, event, and booking records. A local persistence component is intended to save and reload application state.

```
Operator -> CampusConsole (UI) -> Service Layer
                                  StudentService
                                  EventService
                                  ReportService
                                          |
                                          v
                              Model: Student/Event/Booking
                                          |
                                          v
                              AppDataStore / local files
```

7. Design Diagrams

7.1 Workflow

```
Start -> Load data -> Display menu -> Select operation
  -> Register student -> Validate -> Save -> Confirmation
  -> Create event ----> Validate -> Save -> Confirmation
  -> Book seat --------> Check IDs/capacity -> Save -> Booking ID
  -> Cancel booking ---> Update status/seats -> Save
  -> List/report ------> Read state -> Display
  -> Exit
```

7.2 Use Cases

Primary actor: campus staff or console operator. Use cases: register/list students, create/list events, book/cancel seats, view bookings, and generate reports.

7.3 Sequence — Seat Booking

```
Operator -> CampusConsole: choose Book seat
CampusConsole -> StudentService: validate student
CampusConsole -> EventService: validate event/capacity
EventService -> Booking workflow: create booking/update seats
Booking workflow -> AppDataStore: save state
CampusConsole -> Operator: booking ID or error
```

7.4 Class/Component Diagram

```
CampusConsole -> StudentService -> Student
CampusConsole -> EventService -> Event
CampusConsole -> ReportService
Booking workflow -> Booking
Services -> AppDataStore
```

7.5 Conceptual Storage/ER Diagram

```
STUDENT (studentId PK) 1 --- M BOOKING M --- 1 EVENT (eventId PK)
Student: name, department, email, academic year
Event: title, date, venue, capacity, available seats
Booking: bookingId, studentId, eventId, status
```

This is a logical data model. If the implementation uses serialized files rather than a relational database, these entities are logical records, not SQL tables.

8. Design Decisions and Rationale

• Console UI keeps the project focused on Java fundamentals.

• Maven standardizes build, tests, and JAR packaging.

• JUnit supports repeatable automated checks.

• Separate model, service, and UI packages improve maintainability.

• Local persistence avoids requiring an external database for a classroom demonstration.

9. Implementation Details

The project uses Java 17 and Maven. The visible source structure includes Student, Event, and Booking model classes; StudentService, EventService, and ReportService; CampusConsole; and StudentServiceTest. The JAR manifest was configured to launch `com.vityarthi.campus.ui.CampusConsole`.

The observed console menu is: 1 Register student; 2 Create event; 3 List students; 4 List events; 5 Book seat; 6 Cancel booking; 7 View my bookings; 8 Generate reports; 0 Exit. Event dates use `yyyy-MM-dd`.

10. Screenshots and Results

Development screenshots showed successful Maven packaging and JAR startup; student registration generated STU-003; event creation generated EVT-003; booking generated BKG-004; cancellation displayed a success message; and reports listed student/event totals and seat availability.

Before submission: insert genuine screenshots from the final build showing the menu, student registration, event creation, booking/cancellation, reports, and the final persistence restart test. Ensure screenshots match the submitted code.

11. Testing Approach

Automated testing

The latest Copilot Agent output reported `mvn clean test` success with 3 tests, 0 failures, 0 errors, and 0 skipped; it also reported `mvn package` success. Re-run these commands on the exact final project before submission.

Manual testing observed

• Student registration and listing worked within one session.

• Event creation and listing worked within one session.

• Seat booking generated BKG-004.

• Cancellation reported success.

• Reports displayed student totals, event totals, bookings, and seat counts.

Outstanding verification

Persistence is not conclusively verified. A manual restart after persistence changes still displayed `No students available`, while the coding agent reported an isolated restart test passing. Do not claim persistence is complete until a newly created record survives exit and restart in the actual project directory.

Additional recommended tests: invalid dates, empty required fields, duplicate booking attempts, full capacity, cancellation of nonexistent/already-cancelled bookings, seat restoration, and restart persistence.

12. Challenges Faced

• Configuring the executable JAR manifest.

• Handling dates in ISO `yyyy-MM-dd` format.

• Keeping booking and seat availability consistent.

• Diagnosing the mismatch between automated persistence tests and manual restart behavior.

13. Learnings and Key Takeaways

• Layering a Java application into models, services, and UI.

• Using Maven for build, testing, and packaging.

• Writing JUnit tests.

• Validating input and presenting clear errors.

• Why integration and restart tests are needed in addition to unit tests.

• Documenting requirements, architecture, workflows, and limitations.

14. Future Enhancements

• Resolve and verify persistence in the actual runtime directory.

• Add update/delete operations for students and events.

• Improve duplicate prevention and booking ownership checks.

• Consider SQLite/MySQL for larger datasets.

• Add authentication and role-based access.

• Add CSV/PDF report export and logging.

• Expand automated edge-case and end-to-end tests.

15. References

• VITyarthi — Build Your Own Project: General Project Instructions & Submission Guidelines (provided brief).

• Oracle Java documentation: https://docs.oracle.com/en/java/

• Apache Maven guides: https://maven.apache.org/guides/

• JUnit 5 User Guide: https://junit.org/junit5/docs/current/user-guide/

GitHub: insert the actual public CampusConnect repository URL before submitting.

Final checklist: resolve persistence, rerun tests/package, insert final screenshots, verify repository URL, proofread, and upload the PDF.